AgentProf: Semantic Profiling for AI Agents

Anonymous submission

Abstract

AI agents orchestrate multi-step activities across users, tools, and system resources. To improve agent quality, safety, and cost efficiency, developers need to determine where failures concentrate, which workflows trigger unsafe effects, and which task categories consume the most budget. Traditional systems profiling answers analogous questions by aggregating resource use over code paths rather than debugging individual executions, but existing agent observability tools do not combine source-linked additive system effects with selectable profiler projections. Agent responsibility lies in semantic categories such as task intent and workflow phase rather than code paths, but trajectories lack stable identifiers and function-call hierarchies for automatic profiling attribution. Agent observability needs profiling, not only debugging. Our *semantic operation stack model* adapts profiling to agent trajectories by representing agent activities as uniform *operations* and replacing runtime stacks with *operation stacks* that attribute the same data hierarchically. AgentProf implements this model as a profiler with pluggable algorithms for *intent attribution* and *stack construction*, compiling local agent trajectories into pprof-compatible profiles. On a fixed suite of 20 real Codex tasks, scoped source lineage reaches 100.0% precision and 96.6% recall, rejects all 1,629 concurrent-control effects, and AgentProf preserves all 1,520 attributed effects during folding. Across real and public trajectories, semantic profiles separate resource use by responsible category. On 287 OSWorld-Human development sessions, the label-free constructor reaches 0.680 boundary and 0.786 B^3 F1 versus 0.645 and 0.678 for simple controls; a supervised comparator reaches 0.739 and 0.816. On 405 source-valid failed trajectories from the CodeTraceBench verified split, post-hoc monotone calibration raises the prior global constructor's B^3 F1 from 0.475 to 0.649 while preserving every OSWorld decision. AgentProf processes 27,765 operations in 1.17 s.

Introduction

AI agents (Yang et al. 2024; OpenAI 2025; Anthropic 2025; Xie et al. 2024) orchestrate multi-step activities across a high-level intent layer (prompts, LLM calls, and tool invocations) and a low-level system-effects layer (process spawns and file and network I/O). As agents evolve from short scripts into complex systems, production teams accumulate many trajectories over weeks or months.

To improve agent quality, safety, and cost efficiency, developers need to analyze operational behavior across trajectories and interactions. Which categories of tasks consume the most token budget? Where do failures concentrate across workflows? Which behavioral patterns trigger unsafe system effects? At scale, manual trajectory inspection is slow and expensive (Lù et al. 2025), while per-trajectory LLM judging requires a separate evaluator pass for every run (Mazaheri and Mazaheri 2026; Zheng et al. 2023). Profiling provides the aggregation method these questions require. Traditional systems profilers attribute resource consumption to responsible entities rather than debug or trace individual executions.

Existing agent tools provide rich tracing, metadata aggregation, and population dashboards (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024; Datadog 2025; Laminar 2025). Some now derive hierarchical categories across traces and aggregate latency, cost, or evaluation metrics (LangChain 2026; Datadog 2026). They do not provide source-linked, additive cross-layer effects as selectable pprof-compatible operation-stack projections.

Adapting profiling to agent trajectories creates two challenges, detailed in Background and Motivation. Traditional profilers use stable function names to identify responsible code paths and runtime call nesting to provide the attribution hierarchy (Google 2024b; Gregg 2011). Agent responsibility instead lies in semantic categories such as task intent and tool operation. Agent trajectories lack both properties. Natural-language prompts provide no stable identifiers for shared intent. Agent events also do not nest like function calls. A prompt sequence may represent one or multiple tasks, and one task may belong to a larger task.

Agent observability needs profiling, not only debugging. Despite these differences, the fundamental profiling methodology transfers to agent trajectories by projecting resources onto responsible entities, assigning stable tags, and attributing resources hierarchically. We propose a *semantic operation stack model* with two components. *Operations* are uniform records with string fields and additive measures that represent all agent activities (prompts, LLM calls, tool invocations, and system effects) without type-specific objects. *Operation stacks* provide hierarchical attribution, replacing the runtime call stack. Different field choices change the at-

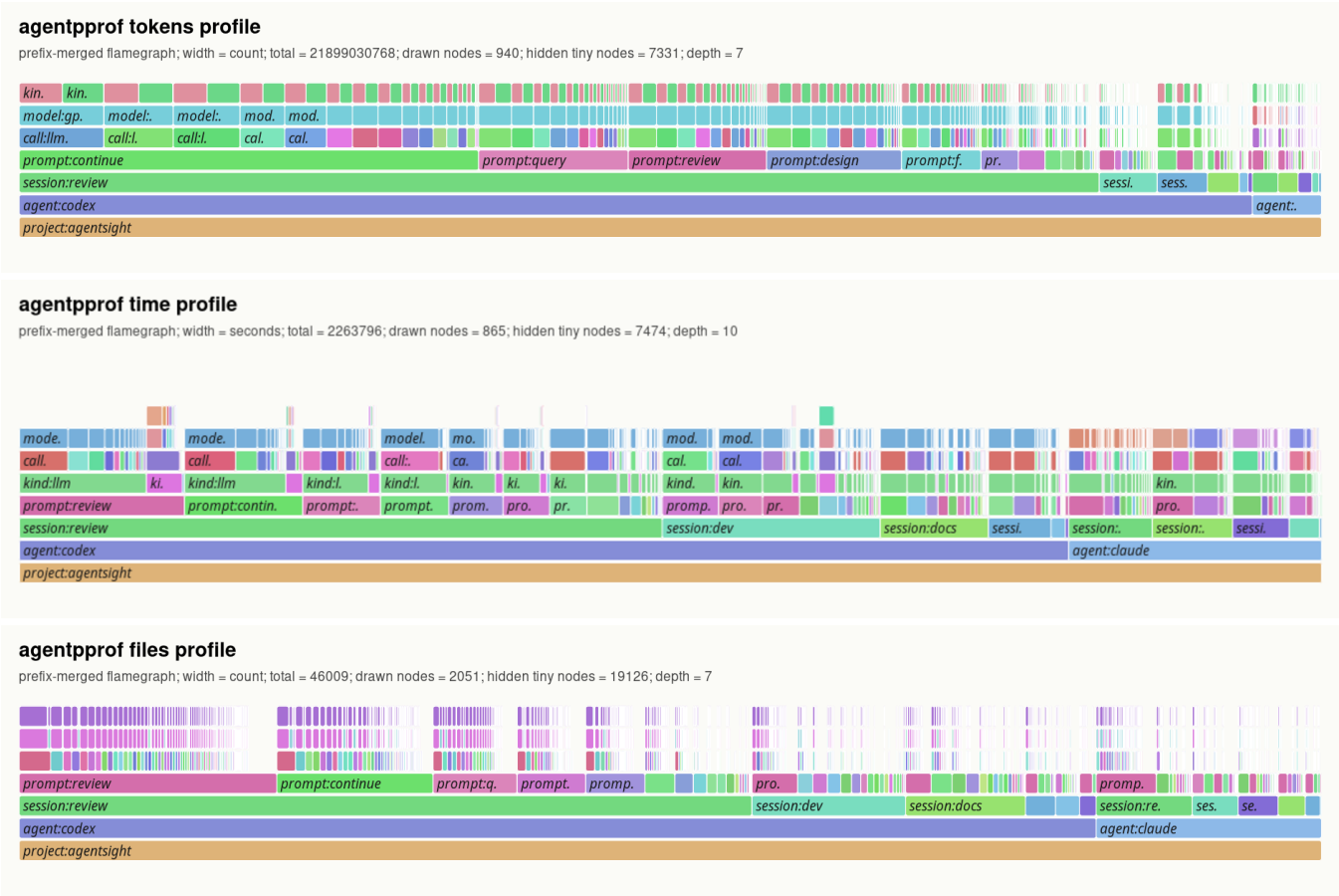

Figure 1: Semantic flame graphs of 325 real agent trajectories under three views. Width represents token count in the top view, wall-clock duration in the middle view, and file-operation count in the bottom view. All three are produced from the same operations by changing only the stack fields and weight function.

tribution granularity. The same operations can be attributed by task category, execution phase, session, or action type (Figure 1).

We implement this model in AgentProf, an offline profiler that compiles local agent trajectories into pprof-compatible profiles (Google 2024b) (Figure 2). Two pluggable mechanisms instantiate the model. *Intent attribution* derives stable, low-cardinality tags from natural-language fields via regex rules, a local LLM tagger, or unsupervised clustering. This gives agents the stable tags that traditional profiling gets for free from function names. *Stack construction* builds the attribution hierarchy from either a user-specified field list or an automatic constructor over visible operations. On 20 real Codex tasks, source lineage reaches 100.0% precision and 96.6% recall, rejects all 1,629 controls, and preserves all 1,520 attributed effects during folding. On 325 real Codex and Claude trajectories, prompt tags reduce mixed system-effect weight from 90.4% to 36.7%; a session view leaves 84.4% mixed. Across three complete public benchmarks with 27,346 labeled steps (Fan et al. 2026; Wang et al. 2026b; Chen et al. 2026a), AP rises from 0.556 to 0.588 on AgentProcessBench; inspection work falls from 46.29% to 41.57% on HINTBench and from 46.64% to 19.55% at TraceElephant's descriptive point. On 287 OSWorld-Human development sessions (Abhyankar, Qi, and Zhang 2025), label-free recurrence reaches 0.680 boundary and 0.786 B^3 F1 versus simple controls at 0.645 and 0.678; a supervised comparator reaches 0.739 and 0.816. On 405 source-valid failed trajectories from the CodeTraceBench verified split (Li et al. 2026), post-hoc monotone calibration raises the prior global constructor's B^3 F1 from 0.475 to 0.649, improves it in all four agent frameworks, and leaves every OSWorld decision unchanged. On four public workloads, AgentProf profiles 27,765 operations in 1.17 s and 464.5 MiB, adding 18.2% time and 1.3% memory over raw action.

This paper makes three contributions:

1. **Semantic operation stack model.** We identify the missing profiling layer in agent observability and propose a model of uniform operations and query-time operation stacks, developed in Background and Motivation and Design.
2. **System: AgentProf.** A profiler that implements this model with pluggable intent attribution and stack construction, producing pprof-compatible output, as detailed

in Implementation.

3. **Evaluation.** We evaluate source lineage on 20 real Codex tasks, resource views on 325 real trajectories and 15 mapped public families, problem correspondence on three complete public benchmarks, boundary accuracy on 287 held-out task instances, post-hoc calibration on 405 source-valid failed CodeTraceBench trajectories, and construction cost on 27,765 operations.

Background and Motivation

LLMs and AI Agents

A typical agent trajectory interleaves two layers of activity (Yang et al. 2024; OpenAI 2025; Anthropic 2025; Zheng et al. 2025). At the intent layer, the agent receives a user prompt, issues LLM calls, and invokes tools for code execution, web search, and file editing. Each tool invocation triggers lower-layer system effects, including process spawns, file reads and writes, and network requests (Zheng et al. 2025). A single trajectory may contain hundreds of such intent-effect cycles, and a production team accumulates many trajectories over time.

System Profiling

Profiling aggregates measurements to attribute resource consumption to responsible entities, in contrast to single-execution debugging and tracing. In traditional software, the profiling pipeline uses three steps: (1) a profiler periodically samples execution state, (2) it attaches each sample to the current call stack, and (3) it merges samples with identical stacks (Google 2024b; Gregg 2011). Flame graphs (Gregg 2011) and pprof (Google 2024b) call graphs visualize aggregate cost as width or node size. Perfetto (Google 2024a) generalizes this with SQL-based analysis and derived events. These tools commonly begin with stable code identifiers and runtime stacks. Pprof can also aggregate labels and promote `tagroot`/`tagleaf` values to pseudo-frames. Agent trajectories instead must first obtain stable semantic responsibility fields and connect them to downstream effects before such profile views exist.

Challenges for Agent Profiling

Existing tools (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) record agent events as per-execution span trees that support single-run debugging. AgentSight (Zheng et al. 2025) bridges intent and system-effect layers by correlating eBPF-captured system events with intercepted LLM traffic, but the resulting recordings still lack aggregate profiling.

Adapting profiling to agent trajectories faces two core challenges. The first is that the responsible entities differ. In traditional software, profiling attributes cost to code paths, but in agent trajectories the relevant entities are semantic categories such as task intent and workflow phase. Existing standards (OpenTelemetry 2024; Arize AI 2024b) represent application-supplied attributes in spans, but natural-language histories do not automatically provide stable responsibility categories because prompts or tool invocations expressing the same intent may use different strings.

The second challenge is that agent trajectories have no runtime call hierarchy that automatically supplies semantic attribution. In CPU profiling, the call stack determines attribution at every level. For example, `malloc`'s cost belongs simultaneously to `parse`, `process`, and `main`. A prompt sequence may constitute one task or several. A task may also belong to a larger workflow, yet no runtime mechanism marks these boundaries or determines the appropriate attribution granularity. We quantify this in RQ1 in Evaluation.

Design Requirements for Agent Profiling

Traditional profiling relies on three mechanisms that the call stack provides automatically. Agent profiling needs all three but must achieve them without a call stack:

D1: Cross-layer resource projection. In traditional profiling, each sample is taken inside a call context, so resource consumption is automatically attributed to the code path that caused it. Agent activities span an intent layer of prompts and LLM calls and a system-effects layer of file I/O and process spawns. The profiler must connect system-layer resource consumption to intent-layer semantic categories.

D2: Stable tags for folding. In traditional profiling, function names are stable identifiers that enable folding identical stacks. Agent prompts are natural language, so the profiler must derive short, stable, repeatable tags before folding.

D3: Hierarchical attribution. In traditional profiling, the runtime call stack provides the attribution hierarchy. Function calls nest, and folding identical stacks produces the tree that profiles visualize. Agent span trees do not encode semantic responsibility through function-call nesting, so the profiler must construct attribution hierarchies from the data.

Design

Cross-layer resource projection, stable tags, and hierarchical attribution drive the design. Operations, defined in the Semantic Operation Stack Model, satisfy D1 by representing intent and system-effect activities in a single record with tag inheritance, so intent-layer tags propagate to the system effects they trigger. *Intent attribution*, described under Algorithms, satisfies D2 by deriving stable tags from natural-language fields. *Operation stacks* satisfy D3. They replace the runtime call stack with query-time field projections that attribute resources at multiple granularities, allowing the same data to support multiple views without rebuilding the input.

The profiling pipeline has four stages: (1) parse raw trajectories into operations, (2) enrich operations via intent attribution or mapping rules, (3) project each operation onto a stack frame sequence chosen at query time, and (4) fold operations with identical stacks, summing their weights.

Semantic Operation Stack Model

Because agent activities span both the intent and system-effect layers described in Background and Motivation, a profiler should not distinguish between them in the data representation. AgentProf therefore represents every activity

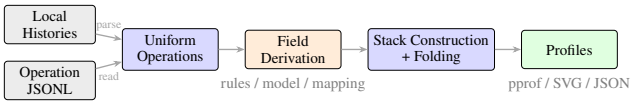

Figure 2: Data-flow and algorithm pipeline. Local histories enter through `agent-session` parsing, while public datasets enter as operation JSONL. Both yield uniform operations before field derivation, stack construction, folding, and profile export.

as an `operation`, a weighted record with string fields and additive measures. Each operation carries string fields (`project`, `agent`, `session`, `prompt_tag`, `kind`, `model`, `path`, `domain`, `status`, ...) and additive measures (token count, duration, event count). A prompt, an LLM call, a tool invocation, a file read, a GUI click, and a process event are all operations with the same schema, distinguished only by which fields carry values. When a tool invocation triggers system effects, ingestion propagates its semantic fields to the resulting operations, allowing downstream resource weights to fold under the responsible intent.

An `operation stack` provides hierarchical attribution for agent trajectories, replacing the runtime call stack. In CPU profiling, the call stack determines which code paths share responsibility for a resource cost. An operation stack serves the same role. An ordered list of fields $[f_1, f_2, \ldots, f_k]$ projects each operation o to a frame sequence $\langle o.f_1, o.f_2, \ldots, o.f_k \rangle$, and operations with identical sequences are merged, summing their weights. Unlike a call stack, the fields are chosen at query time, not determined by execution. The same operations can therefore support a flat task summary, a per-session tree, a semantic phase/action hierarchy, or a trajectory organized by the dataset's own annotations. Changing the fields changes attribution without changing the data. Sessions and spans are optional fields, not hierarchy levels. The same data supports debugging views with `session` and aggregate profiling views without that field. A view is a triple (φ, σ, w). The predicate φ selects participating operations, the stack function $\sigma = [f_1, \ldots, f_k]$ maps each operation to its frame sequence, and the weight function w assigns a nonnegative measure. Four built-in views cover common questions: (1) `tokens` weights LLM calls by token count, (2) `time` weights timed operations by duration, (3) `files` weights file operations by event count, and (4) `network` weights network operations by event count.

Algorithms

Intent attribution. Intent attribution maps natural-language prompts to short, stable, repeatable tags, giving agent trajectories the stable identifiers that traditional profiling gets for free from function names, as discussed in Background and Motivation. The framework supports three pluggable backends. Explicit rules provide repeatable user control, a local model tags natural-language fields, and unsupervised grouping helps discover candidate categories for later rule authoring. Structured trajectories use the same interface to derive fields from existing action and quality attributes. All backends emit ordinary operation fields, keeping the stack model independent of how a tag was obtained.

Stack construction. Stack construction builds attribution hierarchies from available fields. When users supply an ordered list of fields, the projection is direct. When automatic construction is explicitly enabled, AgentProf induces operation stacks without a user-specified field list. It learns which adjacent visible actions reliably recur across reference sessions. Weak or unseen transitions start a new segment; recurring transitions continue the current segment, whose action motif becomes its reusable stack frame. The reference can be a separate label-free operation corpus, so construction does not consume the target session's problem annotations.

Implementation

AgentProf implements the semantic operation stack model as an offline Rust CLI (∼9.8K LOC, Figure 2).

Input reconstruction. AgentProf reads Codex and Claude Code local JSONL histories and operation JSONL. AgentSight (Zheng et al. 2025) recordings first pass through a source-specific adapter that emits supported operations; the current CLI does not read those recordings directly. It then reconstructs prompts, LLM calls, tool invocations, and their linked file and network effects.

Field derivation and boundaries. Regex rules are the production default. Each rule has the form `KIND:TAG=REGEX`, and rules use first-match semantics. Agents can refine the rules through repeated AgentProf runs while inspecting unmatched operations. A local LLM tagger, such as a quantized 3B-parameter model through llama.cpp (Gerganov and contributors 2026), generates a single-word tag per prompt with grammar-constrained decoding and caching. TF-IDF (Salton and Buckley 1988) and K-Means (MacQueen 1967) discover candidate prompt groups without predefined rules, while mapping rules derive fields for structured trajectories, for example `phase:navigate=type:(click|scroll|key)`.

Boundary construction. The Rust inducer counts adjacent action transitions in reference sessions and scores each transition by $\mathrm{NPMI}(a,b) = \ln[p(a,b)/(p_L(a)p_R(b))]/ - \ln p(a,b)$, with all three probabilities defined over the same transition sample space (Bouma 2009). Occurrence-weighted one-dimensional two-means (MacQueen 1967) starts at the minimum and maximum scores, assigns distance ties to the lower center, and uses the converged midpoint as a global cutoff. The same calibration over action-changing occurrences yields a cross-action cutoff. Same-action pairs use the global cutoff; action-changing pairs use the smaller of the two, so the refinement can remove but never add a global-rule boundary. Unseen transitions or scores below the applied cutoff start a new segment. Run-length-compressed action sequences name the resulting recurring motifs. The inducer reads only session identity, input order, and action; scoring labels and resource weights do not affect construction. An optional supervised mode keeps the NPMI score but selects one scalar cutoff that maximizes per-operation B^3 F1 on disjoint reference groups; it enumerates score intervals,

resolves exact ties toward the smallest cutoff, and never reads target groups. Label-free two-means remains the default.

Profile export. Output formats are pprof protobuf for `go tool pprof -http`, folded stacks for `flamegraph.pl`, standalone SVG flame graphs, and JSON.

Evaluation

Research questions. The evaluation answers four fixed paper-level questions: **RQ1:** Does semantic profiling improve resource attribution? **RQ2:** Does profiler output correspond to real problems? **RQ3:** How accurate are the tags? **RQ4:** What is the profiling cost? Each following subsection states its protocol, reports the corresponding evidence, and gives the strongest answer supported by that evidence.

We use three classes of data. The real-agent dataset comprises 325 readable trajectories (198 Codex, 50 Claude, and 77 Claude subagent) and 183,714 system observations from a multi-month development project. The public annotated dataset comprises 15 real agent or human execution families converted by mapping rules into 47,590 operations. These cover web agents (WebLINX (Lù, Kasner, and Reddy 2024), Mind2Web (Deng et al. 2023), AgentTrek (Xu et al. 2025), WebShop (Yao et al. 2022)), API agents (API-Bank (Li et al. 2023), ToolBench (Qin et al. 2024), tau-bench (Yao et al. 2025)), coding agents (SWE-agent (Yang et al. 2024)), and mobile/GUI agents (AndroidCtrl (Li et al. 2024), GUI-Odyssey (Lu et al. 2025), ScaleCUA (Liu et al. 2026b)). Four mapped families provide structured quality or boundary annotations, including AgentRewardBench (Lù et al. 2025), SATraj-OS (Chen et al. 2026b), AgentNet (Wang et al. 2025), and OSWorld-Human (Abhyankar, Qi, and Zhang 2025). AgentProcessBench, HINTBench, and TraceElephant form the public problem-localization set and provide the independent targets for RQ2 (Fan et al. 2026; Wang et al. 2026b; Chen et al. 2026a). Of HINTBench's reported 629 trajectories, the current official test snapshot enumerates 536. We use all 536 after selecting the field order on the separate 80-trajectory validation snapshot (Wang et al. 2026b).

RQ1: Resource Attribution

RQ1 asks whether semantic profiling improves resource attribution beyond what traditional tag-free tools provide. We first test whether a source-linked path assigns system effects to a declared agent process/tool lineage while rejecting concurrent controls. The fixed suite runs 20 real Codex tasks and one concurrent negative control per task through the existing R114-compatible AgentSight 0.2.37 capture path, then converts the scoped joined effects to operation JSONL and folds them once with the current `agentpprof 0.2.37` binary.

All 20 tasks and controls complete. The scoped join recovers 1,520 of 1,574 in-scope effects with no false positive (100.0% precision, 96.569% recall), and none of 1,629 control effects joins. AgentProf emits 1,520 samples in 152 stacks, exactly preserving the input total and all five manifest-category weights.

This result establishes source fidelity for the suite's declared process/tool scope and lossless folding by AgentProf. The task categories come from the fixed manifest rather than automatic task inference. We run a semantic-axis ablation on 325 real Codex/Claude trajectories (183,714 system observations). The prompt tags were produced by the local 3B-model backend described in Implementation and are treated here as declared categories, not as an independent correctness oracle. We progressively add four tag-axis configurations: (1) no tags, (2) session only, (3) prompt only, and (4) both. We then measure mixed-weight percentage, defined as the fraction of system-effect cost in groups containing observations from multiple task categories. A lower mixed-weight percentage means fewer effects from different prompt-tag categories remain in the same responsibility group. The residual percentage measures weight in groups where the majority category accounts for less than 90% of the group, capturing partial mixing.

Figure 3 shows the result. Without semantic tags, nearly all system-effect cost falls into mixed groups. For example, a flat `cargo test` counter shows 2,903 executions but cannot distinguish whether they came from `review`, `debug`, `refactor`, or `test` prompts. The prompt tag is the decisive axis because it reduces both mixed weight and residual by more than half and nearly doubles the number of unique stacks. It separates observations that share the same system call but originate from different task intents. The session tag alone barely helps because trajectories typically contain multiple intent phases.

A tag-permutation test (1,000 shuffles, $p = 0.001$) confirms the separation is not random. The RQ1 experiment measures attribution conditional on the declared prompt tags. The independent RQ3 experiment evaluates group-boundary accuracy. These results show that prompt-level semantic fields materially refine responsibility views beyond tag-free aggregation.

Beyond tag separation, five field selections fold the same 13,265 annotated operations into 9 dataset, 57 phase, 226 semantic, 455 action, or 3,757 per-session groups. Unlike a single-tag GROUP BY, these depths answer task-budget, action-duplication, and session-detail questions. The same model covers all 15 trajectory families without type-specific objects.

The `time` and `tokens` views of the same 325 trajectories share only 7/10 top prompt categories (Spearman $\rho = 0.623$), and `network`-tagged prompts rank 8th by time but 93rd by tokens. Thus, multiple weights expose bottlenecks that one metric would miss.

Figure 1 shows the resulting flame graphs for the `tokens` and `files` views, illustrating that changing only the stack fields and weight function produces different aggregates from the same operations. Together, these results answer RQ1. Scoped source lineage rejects concurrent controls, AgentProf preserves attributed weight across runs, and semantic profiling provides multi-resolution and multi-weight attribution views over the resulting operations. RQ3 separately evaluates automatic group construction.

RQ2: Problem Correspondence

RQ2 asks whether target-blind profiles place independently annotated problems in groups a developer can inspect early.

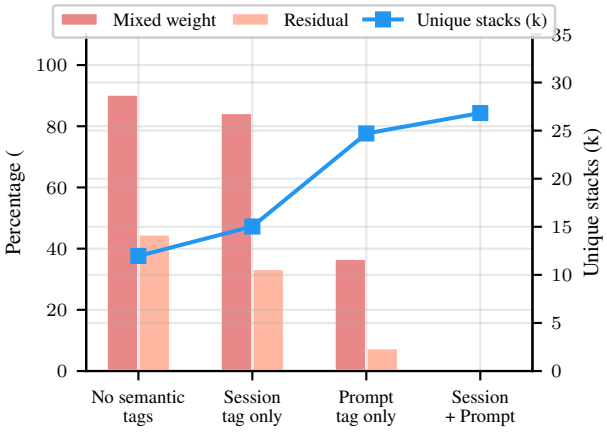

Figure 3: RQ1 semantic-axis ablation on 183,714 system observations from 325 real trajectories. Bars show mixed weight and residual percentages on the left axis, and the line shows unique stacks on the right axis. Adding prompt tags reduces mixed weight from 90.4% to 36.7% while increasing unique stacks from 12k to 25k. Session tags alone barely help (84.4%).

Metric	AgentProf	Raw	Other existing views
AgentProcess AP	.588	.556	flat .325; session .599; step .777
HINT Work@80	.416	.463	native .579; session .591; step 1.00
Trace Work@80	1.00	.719	width .677; native/session 1.00
Trace Work@50†	.195	.466	width .396; session .568; native .578

Table 1: Full-workload RQ2 controls. Lower Work is better; † is descriptive. AP and Work are not averaged.

We run complete AgentProcessBench, HINTBench, and TraceElephant workloads (Fan et al. 2026; Wang et al. 2026b; Chen et al. 2026a). HINTBench development labels select one of 24 fixed field orders; all reported test targets remain hidden from construction and ranking, then the final scorer computes inspection work or macro AP. AgentProcessBench ranks the mean of 20 released judge votes; HINTBench and TraceElephant rank released localization signals by the best 95% Wilson lower bound over profile prefixes (Wilson 1927). Raw action changes only the grouping field.

A fixed quantized Qwen3.6-27B reader (Qwen Team 2026) selects three of each view's query-aware top five groups on six tasks. Rank, view, and IDs are hidden; all positions are balanced; hidden positives are read only after all 66 responses, then rotation scores are averaged.

On all 1,000 AgentProcessBench trajectories (8,509 labeled steps), macro AP rises from 0.556 to 0.588 (gain 0.0315; 95% interval [0.0151, 0.0535]). A matched within-raw-action shuffle gives $p = 0.00995$; finer session and per-step views still preclude uniform dominance.

On all 536 HINTBench tests, Work@80 is 41.57% versus 46.29% raw; only the raw interval crosses zero. On all 220 TraceElephant failures, Work@50 is 19.55% versus 46.64% raw, but a tied tail requires full work at prospective 80% recall, where the interval crosses zero.

Together these complete workloads provide positive RQ2 evidence. The reader improves recall on 5/6 tasks (median paired delta 8.06 points) and precision on 4/6 (3.55 points) at three groups. Work is higher on 4/6, so this supports group prioritization, not lower work, reader-only causality, human utility, or universal dominance.

RQ3: Tag Accuracy

RQ3 asks how accurately categorical operation fields recover independent annotations. RQ3 covers task, phase, and action labels produced by intent attribution or mappings, as well as group boundaries used to construct the hierarchy. We evaluate task partitions and group boundaries; phase, action, and literal tag names remain outside the current evidence.

We hypothesize that pre-specified target-blind taggers or mappings recover accurate, stable task, phase, and action labels and group boundaries on unseen agent and task families. Each component is scored against its corresponding independent annotations and then profiled with the original additive weights.

We test all 287 OSWorld-Human task-instance sessions (Abhyankar, Qi, and Zhang 2025) with complete group annotations, covering 3,978 operations, 3,691 adjacent pairs, and 2,042 groups. For the supervised Bernoulli Naive Bayes predictor (McCallum and Nigam 1998), we fixed the model family, nine input fields, adjacent-pair feature construction, and training procedure before evaluation. Each session-blocked fold fits parameters and a threshold only on training sessions, then predicts every held-out session once. The predicted boundaries define an ordinary group field that AgentProf projects and folds through its released profile-construction path. We also run the built-in recurrence inducer with the same five session folds: each held-out fold uses only the other sessions' visible action transitions as its label-free reference. Because the recurrence principle was selected after inspecting earlier results on this corpus, we report it as mechanism-development evidence rather than independent cross-family confirmation. The optional reference-calibrated mode uses those same training-fold operations and their group annotations to fit one scalar; the held-out fold contributes only visible actions until predictions are fixed. The three simple controls place a boundary after every operation, whenever the visible action field changes, or whenever the visible phase field changes. We report boundary precision, recall, and F1, plus operation-weighted B^3 partition F1 (Bagga and Baldwin 1998), whose per-operation predicted–human group overlap captures merging and fragmentation.

The supervised predictor reaches 0.739 boundary and 0.816 B^3 F1 versus 0.645 and 0.678 for the strongest controls, leading every fold. AgentProf projects all 3,978 operations into 2,249 weight-conserving stacks. Label-free recurrence reaches 0.680 boundary and 0.786 B^3 F1, above the controls and below the supervised predictor; Rust matches all 3,691 decisions and conserves all 3,978 weights. Reference calibration reaches 0.734 boundary and 0.801 B^3 F1 under the same folds. It measures the value of additional group annotations, not equal-information superiority over the label-free default.

	ANet incorrect	ANet redundant	AReward loop	AReward effect	OSWorld boundary	SATraj unsafe	Median
ΔRecall	-.005	+.008	+.361	+.064	+.357	+.097	**+.081**
ΔPrecision	+.010	+.061	-.105	+.159	-.177	+.369	**+.036**
ΔWork	-.002	+.002	+.294	-.001	+.377	+.011	+.006

Table 2: Rank-hidden reader deltas (stack minus session) at three groups; higher recall/precision and lower work are better.

Method	Prec.	Recall	Boundary F1	B^3 F1
Supervised predictor	0.700	0.782	**0.739**	**0.816**
Calibrated recurrence	0.610	0.922	0.734	0.801
Label-free recurrence	0.592	0.799	0.680	0.786
Always boundary	0.476	1.000	0.645	0.678
Action change	0.386	0.626	0.477	0.659
Phase change	0.441	0.268	0.334	0.665

Table 3: Session-held-out RQ3 boundary fidelity on OSWorld-Human. B^3 measures agreement between predicted and human operation partitions.

Workload	Ops	Sem. (s)	Raw (s)	RSS (MiB)
AgentReward	729	0.04	0.03	19.0
Satraj	4,285	0.17	0.14	73.5
OSWorld	6,010	0.25	0.21	108.4
AgentNet	16,741	0.71	0.58	278.4
Union	27,765	1.17	0.99	464.5

Table 4: RQ4 profile-construction cost. Times are three-run medians, and RSS is the largest observed semantic-profile peak.

Post-hoc selection on OSWorld-Human and 405 source-valid failed trajectories from the CodeTraceBench verified split (Li et al. 2026) preserves every OSWorld decision while raising CodeTraceBench boundary F1 from 0.269 to 0.287 and B^3 F1 from 0.475 to 0.649 across all four frameworks. Phase change reaches 0.225 and 0.654. This selects the default implementation, not every RQ3 tag.

Fitting the optional scalar on 483 disjoint solved sessions raises B^3 F1 on the 405 failed sessions from 0.649 to 0.667 while merging 6,897 groups into 5,331; boundary F1 is 0.236. This supports partition fidelity with reference annotations, not every boundary metric or literal tag names.

Target-blind TF-IDF/K-Means reaches operation-weighted V-measure 0.557 on nine Mind2Web sessions (49 operations) and 0.815 on 100 ScienceWorld sessions (2,504 operations) at full coverage, versus 0 for a constant tag (Deng et al. 2023; Wang et al. 2022). It tests partitions, not literal names.

The pre-specified experiments support target-blind task partitions and session-held-out groups across three public families; recurrence separately validates a development-corpus implementation choice.

RQ4: Profiling Cost

RQ4 times the full post-execution parse–construct–fold–serialize path, excluding capture. We run `agentpprof 0.2.37` three times on four complete public workloads and their union, comparing a fixed semantic hierarchy with identical-input raw action on a 24-core Intel Core Ultra 9 285K with 125 GiB RAM and Linux 6.15.11.

Both time curves are monotonic across the five measured input sizes. The semantic curve has a descriptive slope of 0.0422 ms per operation ($R^2 = 0.9997$), reaching 23,731 operations/s on the union. At that largest input, semantic construction adds 180 ms (18.2%) and 6.0 MiB maximum RSS (1.3%) over raw action.

Together, RQ4 shows practical, predictable construction over the tested range for the current binary.

Scope and Limitations

The evaluation is offline and excludes capture and live-agent overhead. RQ1 is limited to the fixed 20-task suite, declared process/tool scope, and R114-compatible AgentSight 0.2.37 path; manifest categories are inputs. RQ3 establishes task partitions on Mind2Web/ScienceWorld and held-out group boundaries on OSWorld-Human; phase/action need matched annotations and fixed mappings. Its 405-trajectory CodeTraceBench calibration is post-hoc, not independent confirmation.

Related Work

Agent observability tools aggregate traces, metadata, and metrics (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024; Arize AI 2024b; Dong, Lu, and Zhu 2024; Balusu 2026; Wang et al. 2026c; Datadog 2025; Laminar 2025); LangSmith and Datadog add cross-trace hierarchies (LangChain 2026; Datadog 2026), and NeMo aggregates token, latency, bottleneck, and concurrency statistics (NVIDIA 2026). AgentProf instead provides selectable, conserved pprof projections over heterogeneous offline histories and source-linked effects.

Traditional profilers aggregate stacks (Google 2024b; Gregg 2011), while Perfetto queries trace events (Google 2024a); AgentProf derives alternative semantic hierarchies over source-linked operations.

Agent diagnosis localizes failures, explains causes, and develops audit taxonomies (Barke et al. 2026; Wang et al. 2026a; Liu et al. 2026a; Mulian et al. 2026; Mazaheri and Mazaheri 2026); CodeTracer reconstructs state-transition traces (Li et al. 2026). AgentProf instead folds recurring responsibility across trajectories.

Conclusion

Agent observability needs profiling, not only debugging.

References

Abhyankar, R.; Qi, Q.; and Zhang, Y. 2025. OSWorld-Human: Benchmarking the Efficiency of Computer-Use Agents. arXiv preprint arXiv:2506.16042.

Anthropic. 2025. Claude Code: An Agentic Coding Tool. https://code.claude.com/docs/en/overview.

Arize AI. 2024a. Arize Phoenix. https://arize.com/docs/phoenix.

Arize AI. 2024b. OpenInference Specification. https://arize-ai.github.io/openinference/spec/.

Bagga, A.; and Baldwin, B. 1998. Entity-Based Cross-Document Coreferencing Using the Vector Space Model. In *36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics, Volume 1*, 79–85. Montreal, Quebec, Canada: Association for Computational Linguistics.

Balusu, K. C. 2026. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26)*.

Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475.

Bouma, G. 2009. Normalized (Pointwise) Mutual Information in Collocation Extraction. In *From Form to Meaning: Processing Texts Automatically, Proceedings of the Biennial GSCL Conference 2009*, 31–40. Tübingen, Germany.

Chen, M.; Wang, J.; Mu, F.; Wang, Y.; Liu, Z.; Feng, H.; and Wang, Q. 2026a. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-Based Multi-Agent Systems. arXiv preprint arXiv:2604.22708.

Chen, X.; Yin, Z.; He, S.; Huang, B.; Lei, S.; et al. 2026b. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. arXiv preprint arXiv:2605.06230.

Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.

Datadog. 2026. LLM Observability Patterns. https://docs.datadoghq.com/llm_observability/monitoring/patterns/.

Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Dong, L.; Lu, Q.; and Zhu, L. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv preprint arXiv:2411.05285.

Fan, S.; Ye, X.; Huo, Y.; Chen, Z.-Y.; Guo, Y.; Yang, S.; Yang, W.; Ye, S.; Chen, J.; Chen, H.; Cong, X.; and Lin, Y. 2026. AgentProcessBench: Diagnosing Step-Level Process Quality in Tool-Using Agents. In *Proceedings of KDD*.

Gerganov, G.; and contributors. 2026. llama.cpp: LLM Inference in C/C++. GitHub repository.

Google. 2024a. Perfetto Trace Processor. https://perfetto.dev/docs/analysis/trace-processor.

Google. 2024b. pprof. https://github.com/google/pprof.

Gregg, B. 2011. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.

Laminar. 2025. Signals: Structured Event Extraction from Traces. https://docs.lmnr.ai/signals/introduction.

LangChain. 2024. LangSmith Observability. https://docs.langchain.com/langsmith/observability.

LangChain. 2026. LangSmith Insights. https://docs.langchain.com/langsmith/insights.

Langfuse. 2024. Langfuse Observability. https://langfuse.com/docs/observability/overview.

Li, H.; Yao, Y.; Zhu, L.; Feng, R.; Ye, H.; Wang, J.; He, Y.; Zou, P.; Zhang, L.; Lei, X.; Huang, H.; Deng, K.; Sun, M.; Zhang, Z.; Ye, H.; and Liu, J. 2026. CodeTracer: Towards Traceable Agent States. *arXiv preprint arXiv:2604.11641*.

Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 3102–3116.

Li, W.; Bishop, W. E.; Li, A.; Rawles, C.; Campbell-Ajala, F.; Tyamagundlu, D.; and Riva, O. 2024. On the Effects of Data Scale on UI Control Agents. In *NeurIPS 2024, Datasets and Benchmarks Track*.

Liu, Y.; Zhang, C.; Han, Z.; Liu, H.; Wang, Y.; Yu, Y.; Wang, X.; and Yin, Y. 2026a. TrajAD: Trajectory Anomaly Detection for Trustworthy LLM Agents. arXiv preprint arXiv:2602.06443.

Liu, Z.; Xie, J.; Ding, Z.; Li, Z.; Yang, B.; et al. 2026b. ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data. In *Proceedings of ICLR*.

Lu, Q.; Shao, W.; Liu, Z.; Meng, F.; Li, B.; Chen, B.; Huang, S.; Zhang, K.; Qiao, Y.; and Luo, P. 2025. GUIOdyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

Lù, X. H.; Kasner, Z.; and Reddy, S. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. In *Proceedings of ICML*, volume 235 of *Proceedings of Machine Learning Research*, 33007–33056. PMLR.

Lù, X. H.; Kazemnejad, A.; Meade, N.; Patel, A.; Shin, D.; Zambrano, A.; Stańczak, K.; Shaw, P.; Pal, C. J.; and Reddy, S. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. In *Proceedings of COLM*.

MacQueen, J. B. 1967. Some Methods for Classification and Analysis of Multivariate Observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, 281–297. University of California Press.

Mazaheri, P.; and Mazaheri, K. 2026. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. arXiv preprint arXiv:2605.20530.

McCallum, A.; and Nigam, K. 1998. A Comparison of Event Models for Naive Bayes Text Classification. In *AAAI-98 Workshop on Learning for Text Categorization*, 41–48. AAAI Press.

Mulian, H.; Zeltyn, S.; Levy, I.; Galanti, L.; Yaeli, A.; and Shlomov, S. 2026. AgentFixer: From Failure Detection to Fix Recommendations in Agentic Systems. In *AGENT@ICSE*.

NVIDIA. 2026. Profiling and Performance Monitoring of NVIDIA NeMo Agent Toolkit Workflows. Official documentation.

OpenAI. 2025. OpenAI Codex CLI. GitHub repository.

OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. https://github.com/open-telemetry/semantic-conventions-genai.

Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; Zhao, S.; Hong, L.; Tian, R.; Xie, R.; Zhou, J.; Gerber, M.; Li, D.; Liu, Z.; and Sun, M. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

Qwen Team. 2026. Qwen3.6-27B: Flagship-Level Coding in a 27B Dense Model. Official model card.

Salton, G.; and Buckley, C. 1988. Term-Weighting Approaches in Automatic Text Retrieval. *Information Processing and Management*, 24(5): 513–523.

Wang, J.; Feng, Z.; Wu, J.; Li, R.; Xie, Q.; Ren, Y.; Zhu, H.; Han, X.; Meng, F.; Feng, J.; and Liu, J. 2026a. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. arXiv preprint arXiv:2606.02060.

Wang, J.; Hou, J.; Wang, F.; Jian, P.; Bao, C.; and Lv, Z. 2026b. HINTBench: Horizon-Agent Intrinsic Non-Attack Trajectory Benchmark. arXiv preprint arXiv:2604.13954.

Wang, R.; Jansen, P.; Côté, M.-A.; and Ammanabrolu, P. 2022. ScienceWorld: Is Your Agent Smarter than a 5th Grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 11279–11298. Association for Computational Linguistics.

Wang, X.; Wang, B.; Lu, D.; Yang, J.; Xie, T.; et al. 2025. AgentNet. https://huggingface.co/datasets/xlangai/AgentNet.

Wang, Y.; Zhang, J.; Cai, T.; Liu, Z.; Sun, Q.; Sun, Z.; Wu, Z.; Dong, M.; Zheng, M.; Yin, X.; and Zhu, Y. 2026c. From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents. arXiv preprint arXiv:2606.04990.

Wilson, E. B. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158): 209–212.

Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shin, D.; Lei, F.; Liu, Y.; Xu, Y.; Zhou, S.; Savarese, S.; Xiong, C.; Zhong, V.; and Yu, T. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.

Xu, Y.; Lu, D.; Shen, Z.; Wang, J.; Wang, Z.; Mao, Y.; Xiong, C.; and Yu, T. 2025. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. In *Proceedings of ICLR*.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2025. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *Proceedings of ICLR*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Zheng, Y.; Hu, Y.; Yu, T.; and Quinn, A. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*.